```
import pandas as pd
import numpy as np

from xgboost import XGBRegressor

from sklearn.metrics import mean_squared_error
from sklearn.metrics import mean_absolute_error

import matplotlib.pyplot as plt
import seaborn as sns
```

```
train = pd.read_csv("train_final.csv")
test = pd.read_csv("test_final.csv")

print(train.shape)
print(test.shape)
```

```
(20631, 21)
(13096, 21)
```

```
train["RUL"] = train["RUL"].clip(upper=125)
test["RUL"] = test["RUL"].clip(upper=125)
```

```
print(train["RUL"].max())
```

```
125
```

```
top_sensors = [
    "s11",
    "s12",
    "s6",
    "s4",
    "s7"
]
```

```
for sensor in top_sensors:

    train[f"{sensor}_roll5"] = (
        train.groupby("unit_id")[sensor]
        .transform(
            lambda x: x.rolling(
                window=5,
                min_periods=1
            ).mean()
        )
    )

    test[f"{sensor}_roll5"] = (
        test.groupby("unit_id")[sensor]
        .transform(
            lambda x: x.rolling(
                window=5,
                min_periods=1
            ).mean()
        )
    )
```

```
for sensor in top_sensors:

    train[f"{sensor}_lag1"] = (
        train.groupby("unit_id")[sensor]
        .shift(1)
    )

    test[f"{sensor}_lag1"] = (
        test.groupby("unit_id")[sensor]
        .shift(1)
    )
```

```
for sensor in top_sensors:
```

```
    train[f"{sensor}_delta"] = (
        train[sensor]
        - train[f"{sensor}_lag1"]
    )

    test[f"{sensor}_delta"] = (
        test[sensor]
        - test[f"{sensor}_lag1"]
    )
```

```
train = train.fillna(0)
test = test.fillna(0)
```

```
test = test.dropna(subset=["RUL"])
```

```
FEATURES = [
    c for c in train.columns
    if c not in ["unit_id","RUL"]
]

X_train = train[FEATURES]
y_train = train["RUL"]

X_test = test[FEATURES]
y_test = test["RUL"]
```

Double-click (or enter) to edit

```
model = XGBRegressor(
    n_estimators=500,
    max_depth=6,
    learning_rate=0.05,
    subsample=0.8,
    colsample_bytree=0.8,
    random_state=42,
    n_jobs=-1
)

model.fit(X_train, y_train)
```

```
XGBRegressor
XGBRegressor(base_score=None, booster=None, callbacks=None,
             colsample_bylevel=None, colsample_bynode=None,
             colsample_bytree=0.8, device=None, early_stopping_rounds=None,
             enable_categorical=False, eval_metric=None, feature_types=None,
             feature_weights=None, gamma=None, grow_policy=None,
             importance_type=None, interaction_constraints=None,
             learning_rate=0.05, max_bin=None, max_cat_threshold=None,
             max_cat_to_onehot=None, max_delta_step=None, max_depth=6,
             max_leaves=None, min_child_weight=None, missing=nan,
             monotone_constraints=None, multi_strategy=None, n_estimators=500,
             n_jobs=-1, num_parallel_tree=None, ...)
```

```
preds = model.predict(X_test)
```

```
rmse = np.sqrt(mean_squared_error(y_test, preds))
mae = mean_absolute_error(y_test, preds)

print("RMSE =", rmse)
print("MAE =", mae)
```

```
RMSE = 43.81989758168175
MAE = 32.9712795065147
```

```
importance = pd.DataFrame({
    "Feature": FEATURES,
    "Importance": model.feature_importances_
})
```

```
importance = importance.sort_values(
    "Importance",
    ascending=False
)

importance.head(15)
```

	Feature	Importance
19	s11_roll5	0.480121
20	s12_roll5	0.245998
22	s4_roll5	0.140019
23	s7_roll5	0.037962
0	cycle	0.013172
14	s14	0.009150
26	s6_lag1	0.008309
10	s9	0.006997
7	s6	0.006321
21	s6_roll5	0.004940
15	s15	0.004567
16	s17	0.004145
17	s20	0.003390
13	s13	0.002899
18	s21	0.002702

```
print(X_train.shape)
print(X_test.shape)
```

```
(20631, 34)
(13096, 34)
```

```
TOP10 = [
's11_roll5',
's12_roll5',
's4_roll5',
's7_roll5',
'cycle',
's14',
's6_lag1',
's9',
's6',
's6_roll5'
]
```

```
X_train = train[TOP10]
X_test = test[TOP10]

y_train = train["RUL"]
y_test = test["RUL"]
```

```
model = XGBRegressor(
    n_estimators=500,
    max_depth=6,
    learning_rate=0.05,
    subsample=0.8,
    colsample_bytree=0.8,
    random_state=42,
    n_jobs=-1
)

model.fit(X_train, y_train)
```

```
XGBRegressor
XGBRegressor(base_score=None, booster=None, callbacks=None,
             colsample_bylevel=None, colsample_bynode=None,
             colsample_bytree=0.8, device=None, early_stopping_rounds=None,
             enable_categorical=False, eval_metric=None, feature_types=None,
             feature_weights=None, gamma=None, grow_policy=None,
             importance_type=None, interaction_constraints=None,
             learning_rate=0.05, max_bin=None, max_cat_threshold=None,
             max_cat_to_onehot=None, max_delta_step=None, max_depth=6,
             max_leaves=None, min_child_weight=None, missing=nan,
             monotone_constraints=None, multi_strategy=None, n_estimators=500,
             n_jobs=-1, num_parallel_tree=None, ...)
```

```
preds = model.predict(X_test)

rmse = np.sqrt(mean_squared_error(y_test, preds))
mae = mean_absolute_error(y_test, preds)

print("RMSE =", rmse)
print("MAE =", mae)
```

```
RMSE = 43.160858782092774
MAE = 31.34428317392718
```

```
import joblib

joblib.dump(model, "xgb_rul_model.pkl")

print("Saved successfully")
```

```
Saved successfully
```

```
train.to_csv("train_engineered.csv", index=False)
test.to_csv("test_engineered.csv", index=False)
```

```
from google.colab import files

files.download("train_engineered.csv")
files.download("test_engineered.csv")
```

```
%who
```

```
FEATURES         TOP10   XGBRegressor    X_test  X_train         importance      joblib  mae     mean_absolute_error
mean_squared_error       model   np      pd      plt     preds   rmse    sensor  sns
test     top_sensors     train   y_test  y_train
```